

Combinatorial Algorithms (Math 482), 2026 Spring Problem Set 9

Due by Wednesday, April 1, 2026.

1. **Union-find data structure.** In class, we discuss a union-find data structure: A set $S = \{1, 2, \dots, n\}$ is partitioned into k subsets, $S = A_1 \cup A_2 \cup \dots \cup A_k$. Each set A_ℓ is represented by a rooted tree T_ℓ , for each element $i \in A_\ell$, the operation $\text{FIND}(i)$ returns the element at the root of T_ℓ . For two elements, $i, j \in S$, if $\text{FIND}(i) \neq \text{FIND}(j)$, then the $\text{UNION}(i, j)$ operation merges the trees that contain i and j , by adding an edge from the root of the smaller tree to the root of the larger tree (ties are broken arbitrarily).
 - (a) Work an example, where $S = \{1, 2, \dots, 7\}$, the initial partition consists of 1-element sets. Simulate the data structure for the following sequence of operations: $\text{Union}(1, 3)$, $\text{Union}(1, 7)$, $\text{Union}(2, 5)$, $\text{Union}(4, 6)$, $\text{Union}(2, 3)$, $\text{Find}(5)$.
 - (b) Prove, by induction on $t \geq 0$, that if a tree T_ℓ has 2^t or more vertices, then its height is at most t . (The *height* of the tree is the maximum distance between a leaf and the root.)
 - (c) Now suppose that we modify the data structure and the $\text{UNION}(i, j)$ operation can add an edge between $\text{FIND}(i)$ and $\text{FIND}(j)$ in any direction. Construct an example for a 7-element set and 6 union operations that produce a tree of height 5.
2. **Dijkstra's algorithm.**
 - (a) Create a directed graph $G = (V, E)$ with 12 vertices and 24 randomly selected edges. Assign weights 1, 2, 3, 4, 5 to the edges randomly. Choose a random source s , and run Dijkstra's algorithm to find all single-source shortest paths from s .
 - (b) Draw a copy of the graph with the same weights and the same source s as in part (a), but assume that the edges are undirected now. Run Prim's algorithm to find an MST.
 - (c) Construct an example for an edge-weighted directed graph $G = (V, E)$ with 6 vertices where some of the edge weights are negative, and Dijkstra's algorithm does not find the shortest paths from s . (In your example, show what Dijkstra's algorithm returns, and explain why it is not the correct shortest path.)
3. **Bellman-Ford algorithm.** We are given an directed graph $G = (V, E)$, a source s , and arbitrary edge weights $w(e)$. Recall that the Bellman-Ford algorithm initializes $d(s) = 0$ and $d(v) = \infty$ for all $v \neq s$. The algorithm consists of two nested for-loops: The outer loop repeats $n - 1$ times; and in the inner loop, for each directed edge (u, v) , we put $d(v) \leftarrow \min(d(v), d(u) + w(u, v))$.
 - (a) Prove that if G contains a negative cycle, then at the end of the Bellman-Ford algorithm, there will be an edge (u, v) such that $d(u) + w(u, v) < d(v)$.
 - (b) Construct an example for an edge-weighted directed graph $G = (V, E)$ with 6 vertices and 10 edges such that: If we repeat the outer loop of the Bellman-Ford algorithm only 4 times, then $d(v)$ is not yet the true distance to s for some vertices.